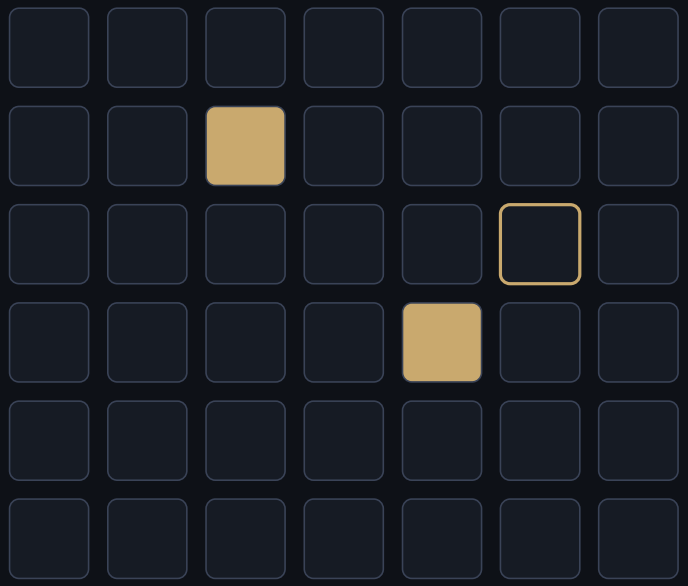
ExamGrid

Data structures in practice

An examination-centre and seating simulator in C++, built on an array, a binary search tree, a circular queue, stacks and a linked list, all implemented by hand.

Group members:

1. **RISHABH MEHTA – 70022500281- B237**
2. **AVIRAJ MAHAJAN-70022500492-B254**



Educational simulation · synthetic data only

PROJECT OVERVIEW

WEBSITE ▸ ALL PAGES

What ExamGrid does

A large entrance exam must seat thousands of candidates in limited halls, admit them in order, and recover when a centre fails. ExamGrid simulates that workflow, and each stage is driven by a data structure written from scratch.

01
Register
Array

Candidate records sit in fixed-capacity arrays.

Website ▸ Overview

02
Index
Binary search tree

Application IDs are indexed for fast lookup and sorted listing.

Website ▸ Candidates

03
Allocate
2D array

Candidates are placed into hall seat grids by preference.

Website ▸ Seating plan

04
Check in
Circular queue + stack

Candidates are admitted first in, first out; the last check-in can be undone.

Website ▸ Operations

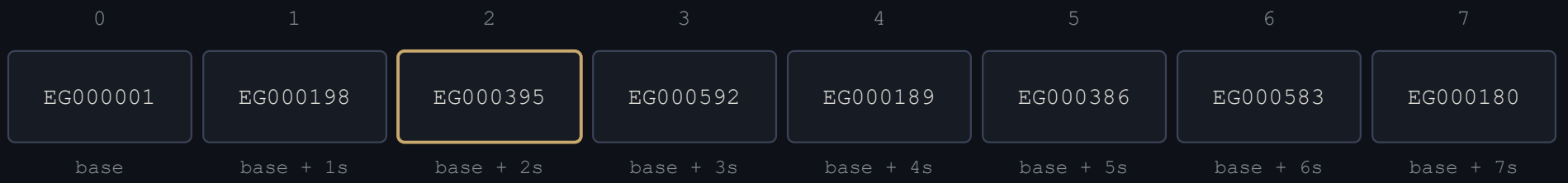
05
Recover
Stack + linked list

A failed centre is re-seated; every change can be undone and is logged.

Website ▸ Operations, Audit trail

WEBSITE ▸ OVERVIEW

Arrays: contiguous memory, constant-time access




```
address of element i  =  base  +  i × sizeof(element)
```

Strengths

Any element is reached in one step by its index. Elements sit side by side, so memory use is predictable.

Costs

Capacity is fixed in advance. Inserting or deleting in the middle shifts every later element: O(n).

In ExamGrid

Candidate[10000] and Centre[12] hold every record. The IDs above are the real first entries of the demo array.

WEBSITE ▸ SEATING PLAN

Two-dimensional arrays: a hall is a grid

	0	1	2	3	4	5	6
0	0	1	2	3	4	5	6
1	7	8	9	10	11	12	13
2	14	15	16	17	**18**	19	20
3	21	22	23	24	25	26	27
4	28	29	30	31	32	33	34

Number inside each cell = position in memory

```
seats[2][4]  →  offset 2 × 7 + 4 = 18
```

Row-major layout

Rows are stored one after another, so seats[r][c] is found with one multiplication and one addition.

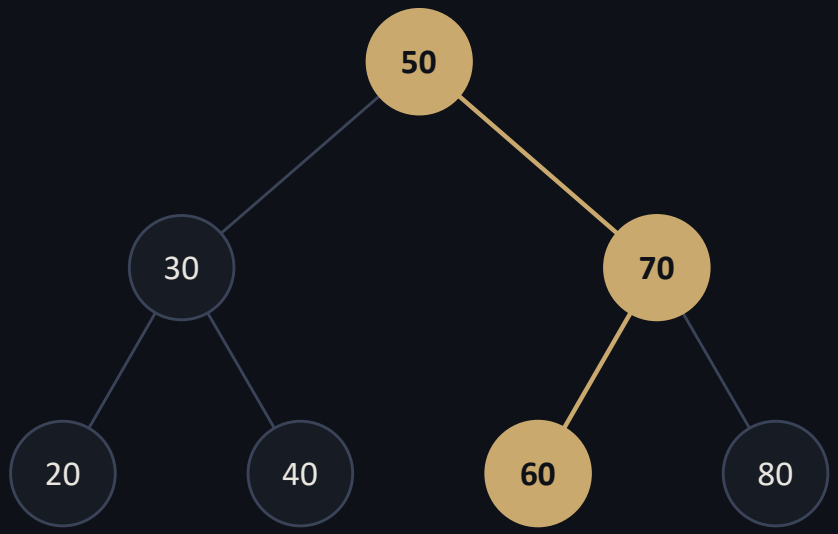
In ExamGrid

Candidate[10000] and Centre[12] hold the records. Every hall has int seats[25][25]; a cell holds a candidate's array index, or −1 when empty. Demo halls are 10 × 15: 900 seats in all.

First-free scan

Allocation walks rows, then columns, and claims the first −1 it finds: O(rows × cols) per candidate.

Binary search tree: ordered nodes



Search for 60: 50 → right → 70 → left → 60

3 comparisons for 7 nodes.

The ordering rule

For every node, all keys in its left subtree are smaller and all keys in its right subtree are larger.

Search and insert

Compare with the node, go left if smaller and right if larger. Search stops at a match or an empty branch; insert attaches a new node there.

Inorder walk and cost

Visit left, node, right: sorted keys in O(n). Search and insert are O(log n) on average, O(n) if the tree is skewed.

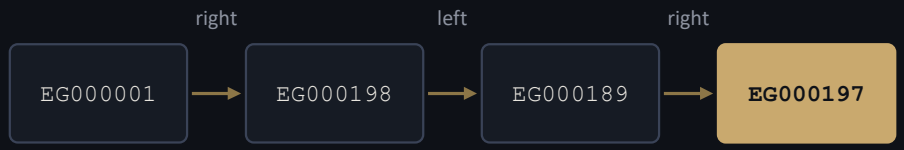
WEBSITE ▸ CANDIDATES

The BST inside ExamGrid

```
struct Node { int candidateIndex;
              int left, right; }; // array indices
Node nodes[MAX_CANDIDATES];

int search(id) {
  int cur = root;
  while (cur != -1) {
    if (id == key(cur)) return cur;
    cur = id < key(cur) ? left : right;
  }
  return -1;
}
```

Search for EG000197 on the Candidates page



4 comparisons across 600 records.

The website reports this comparison count in its DSA panel. The chain shows the path the C++ code walks.

Implementation notes

- Children are array indices, so the tree lives in one array.
- Keys are ID strings compared with < and ==.
- Inorder uses an explicit stack array, not recursion.

Finding on the demo data

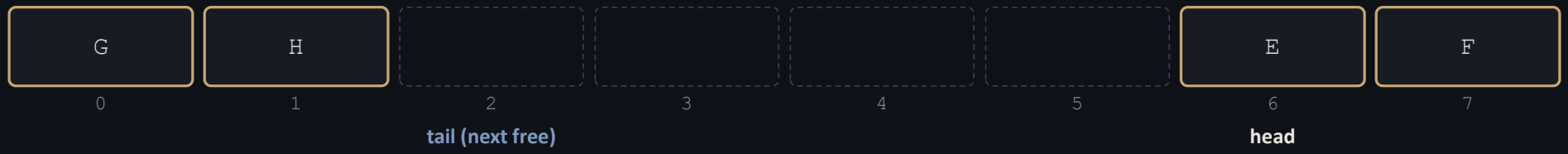
600 nodes, height 33. A balanced tree would need about 10 levels. EG000001 is the first ID inserted and the smallest, so every other ID hangs off the right side of the root.

Circular queue: first in, first out

Linear queue: slots before head are never reused



Circular queue: the tail wraps from index 7 back to 0




```
enqueue(x):  data[tail] = x;
             tail = (tail + 1) % N;  count++
```

```
dequeue():   x = data[head];
             head = (head + 1) % N;  count--
```

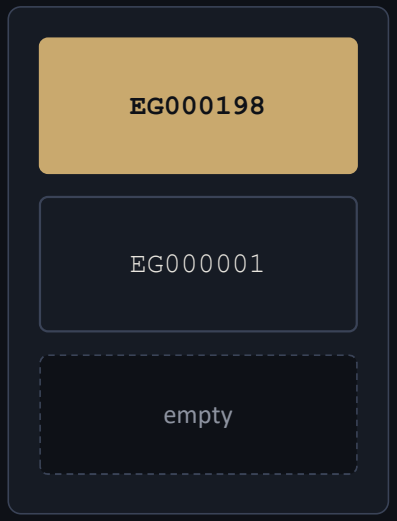
In ExamGrid, N = 10,000: Prepare queue gives head 0, tail 600; two Process next calls give head 2, count 598.

Stacks: last in, first out, in ExamGrid

Undo stack · depth 2 of 5



Check-in history



Centre failure, step by step

1. Push a snapshot of every assignment and centre status.
2. Mark the centre offline and clear its seats.
3. Reallocate displaced candidates by preference.
4. Report: 200 displaced, 200 reallocated, 0 left over.
5. Undo pops the snapshot and rebuilds the seat grids.

A stack only touches its top, so push and pop are O(1). That suits undo: the latest change is reversed first. The undo stack keeps the five most recent snapshots and drops the oldest; restoring one costs O(n + occupied seats) because every seat grid is rebuilt.

WEBSITE ▸ AUDIT TRAIL

Linked list: the audit trail

```
struct Node { std::string message;
              Node* next = nullptr; };
Node *head = nullptr, *tail = nullptr;

void append(const std::string& msg) {
  Node* n = new Node{stamp(msg), nullptr};
  if (!head) head = tail = n;
  else { tail->next = n; tail = n; }
  ++count;
}
```

- Each node holds a message and a pointer to the next node; head and tail pointers are kept.
- The tail pointer makes append O(1); reading the k-th event walks from the head: O(k).
- No fixed capacity and nothing shifts. The destructor frees every node.



PUTTING IT TOGETHER

WEBSITE ▸ OPERATIONS · ALLOCATION

Seat allocation across the structures

Preference 1 centre → **Preference 2 centre** → **Any other active centre** → **Unallocated (exception)**

Inside a centre: scan halls, then rows, then seats, and claim the first cell where seats[r][c] == −1.

Array

Holds every candidate and centre record.

Binary search tree

Finds a candidate and the assigned seat by ID.

2D array

Stores exactly who sits where.

Circular queue

Orders check-in, first come first served.

Stack and linked list

Undo recent changes and log every event.

DESIGN CHOICES

Why these structures and not others

Need	Chosen	Alternative	Why it wins
Find a candidate by ID	BST	Scan an unsorted array	O(h) instead of O(n); inorder gives a sorted listing
Model hall seating	2D array	List of seat objects	Layout maps to rows and columns; O(1) seat access
Admit candidates in order	Circular queue	Array with shifting	O(1) at both ends; freed slots are reused
Undo the latest change	Stack	Search a history list	Latest change is on top; push and pop are O(1)
Record events in order	Linked list	Fixed-size array	No capacity limit; append is O(1) with a tail pointer

A hash table also finds an ID in O(1), but it cannot list IDs in sorted order. The BST gives both lookup and ordering.

SUMMARY

Operation costs at a glance

Operation	Structure	Time	Website page
Read a candidate or seat by index	Array, 2D array	O(1)	Overview, Seating
Allocate all candidates	Arrays + 2D grids	worst case O(N × S)	Operations
Search by application ID	BST	avg O(log n), worst O(n)	Candidates
Sorted candidate listing	BST inorder	O(n)	Candidates
Enqueue or dequeue a check-in	Circular queue	O(1)	Operations
Undo the last check-in	Stack	O(1)	Operations
Push or pop an undo snapshot	Stack	O(1); restore O(n + seats)	Operations
Append an audit event	Linked list	O(1)	Audit trail

HONEST ASSESSMENT

Limitations and next steps

Known limitations

- Plain BST with no rebalancing, so worst-case search is O(n); the demo tree has height 33.
- Fixed capacities: 10,000 candidates, 12 centres and 8 halls.
- Allocation is preference-first, first-free-seat, not a global optimum.
- Each undo snapshot copies every candidate's assignment: O(n).

Possible improvements

- An AVL or red-black tree to keep the height near log n.
- A hash table beside the BST for O(1) exact lookup.
- Dynamic arrays that grow on demand.
- Store only changed assignments in undo snapshots.

LIVE DEMO

WEBSITE ▸ ALL PAGES

Demo script: website and slides together

#	Website page	Action	Concept to explain
1	Overview	Read the metrics and centre cards	Candidate array, BST index, capacity
2	Operations	Build seating plan: Run allocation	Preference-first scan of 2D grids
3	Seating plan	Choose a centre and hall; click a seat	2D array: seats[row][col]
4	Candidates	Search or inspect EG000197	BST: 4 comparisons
5	Operations	Prepare queue, then Process next	Circular FIFO queue
6	Operations	Undo last check-in	Stack pop
7	Operations	Simulate centre failure, then Undo	Snapshot stack and reallocation
8	Audit trail	Read the events	Linked list: append at the tail
9	Candidates	Register a candidate	BST insertion

Thank you

Questions and live demo

Array · Binary search tree · Circular queue · Stack · Linked list

